基于 MQTT、ADXL345 与 Qt 的实时传感器数据采集与可视化系统说明文档

摘要

本项目实现了一套基于 Raspberry Pi 5、ADXL345 三轴加速度传感器、Mosquitto 消息代理和 Qt 图形化上位机的实时传感器数据采集与显示系统。系统通过 I^2C 总线完成树莓派与 ADXL345 之间的数据通信，通过 MQTT 协议实现设备侧与上位机之间的消息传输，并通过 Qt 开发桌面可视化界面，用于显示传感器姿态、加速度、CPU 温度、CPU 负载等信息，同时支持运行日志、主题订阅、消息发布、实时曲线与 60 秒滚动统计等功能。除数据采集与显示外，系统还实现了 LED 状态联动控制，可根据传感器倾斜程度点亮不同指示灯，用于验证执行器侧的联动效果。整体系统具有结构清晰、功能完整、可演示性强、便于二次开发等特点，适合作为 MQTT 课程项目、嵌入式系统实验、物联网综合设计和论文原型系统。

1. 项目背景与意义

在物联网系统中，传感器节点、网络通信和上位机可视化通常构成完整的数据链路。传统实验往往只关注单一环节，例如只读取传感器数据，或只完成上位机界面设计，而缺少从“传感器采集—网络传输—图形显示—状态联动”的完整闭环。本项目围绕这一完整链路展开设计，以 ADXL345 为姿态与加速度采集对象，以 MQTT 为轻量级消息协议，以 Mosquitto 为 Broker，以 Qt 为可视化开发平台，构建出一套可真实运行、可直观演示、可扩展复用的实时传感器数据管线。

2. 系统总体架构

本系统由四个核心部分组成，分别是传感器采集层、MQTT 通信层、桌面可视化层和本地执行器层。

传感器采集层部署在 Raspberry Pi 5 上，使用 ADXL345 作为三轴加速度传感器。树莓派通过 I^2C 读取加速度寄存器数据，并进一步计算 pitch 和 roll。与此同时，树莓派还读取本机 CPU 温度和 CPU 负载，使得系统不仅可以显示外部传感器状态，也能够展示设备自身的运行状态。这一部分的核心代码位于 publisher.cpp。

MQTT 通信层基于 Eclipse Paho MQTT C 客户端和 Mosquitto Broker 构建。树莓派终端将采集到的姿态信息、加速度数据、CPU 状态和时间戳封装为 JSON 消息，发布到指定主题；Qt 上位机订阅对应主题后完成解析与显示。系统同时配置了在线状态消息、离线遗嘱消息、保留消息和不同 QoS 等级，以验证协议特性。Qt 侧的 MQTT 连接管理、主题订阅、消息接收和 JSON 发布逻辑实现于 mainwindow.h 和 mainwindow.cpp。

桌面可视化层采用 Qt Widgets 实现，提供连接参数配置、主题订阅、指标选择、实时曲线、运行日志和 60 秒滚动统计等功能。整个 Qt 工程通过 qmake 管理，在 EEN1071Ass2.pro 中可以看到其依赖模块包括 core、gui、widgets、printsupport 和 network，并链接 Paho MQTT C 库。

本地执行器层也运行在树莓派上，通过三个 LED 反映 ADXL345 当前姿态等级。对应程序见 subscriber_led_wiringpi.cpp。这部分除了订阅 MQTT 指定主题外，还会在本地周期性读取传感器姿态，并根据阈值控制不同 LED 点亮，从而体现“采集—判断—执行”的联动效果。

3. 硬件组成与连接方式

本系统的主要硬件包括 Raspberry Pi 5、ADXL345 三轴加速度传感器模块以及三个 LED 指示灯。树莓派作为边缘采集与控制节点，负责与传感器通信并执行本地逻辑；ADXL345 用于测量三轴加速度并计算俯仰角与横滚角；三个 LED 用于表示不同的倾斜等级。

在通信方式上，树莓派与 ADXL345 之间通过 I^2C 总线连接。代码中 ADXL345 默认地址设置为 0x53，总线号为 1，这一点可在 publisher.cpp 中实例化 ADXL345 sensor(1, 0x53); 看到。I^2C 底层访问由 I2CDevice 类实现，它封装了设备打开、寄存器读写、批量读取等功能，对应文件为 I2CDevice.h 和 I2CDevice.cpp。

LED 连接部分使用 WiringPi 库控制 GPIO，代码中 LED_LOW、LED_MED 和 LED_HIGH 分别定义为 WiringPi 编号 0、2 和 3，对应 BCM 17、27 和 22，这些定义位于 subscriber_led_wiringpi.cpp。

4. 软件环境与依赖

Qt 上位机部分运行在 Windows 10/11 x64 环境下。根据仓库根目录 README，Qt 工程使用 Qt 5.14.2、MinGW 7.3 64-bit 工具链、qmake、mingw32-make 和 gdb 进行构建与调试，且默认链接 Eclipse Paho MQTT C 客户端库与 Mosquitto Broker。相关说明见仓库根目录 README.md。

树莓派终端部分运行在 Linux 环境，主要依赖 Paho MQTT C 库、JsonCpp、WiringPi 和 I^2C 相关头文件。由于 raspberry_pi/README.md 仍然保留了较早的“预留目录”说明，而实际代码已经放在 raspberry_pi/程序源码/ 下，因此实际使用时应以该目录中的源码为准，而不是仅参考该 README。

5. 树莓派终端设计

树莓派终端的主要任务是读取传感器、封装消息并发布到 MQTT Broker，同时在必要时发布设备状态消息。

在驱动设计上，ADXL345 类继承自 I2CDevice，封装了 ADXL345 的寄存器操作、原始加速度读取、分辨率与量程配置，以及 pitch 和 roll 的计算过程，定义见 ADXL345.h，实现见 ADXL345.cpp。其中，程序通过读取 DATAX0~DATAZ1 寄存器获得三轴原始数据，再结合量程和分辨率换算为重力分量，并使用反正切函数计算俯仰角和横滚角。

在数据发布上，publisher.cpp 将传感器姿态、原始加速度、CPU 负载、CPU 温度和时间戳封装成 JSON。代码中 build_json() 函数构造的字段包括 pitch、roll、accel_x、accel_y、accel_z、cpu_load、cpu_temp_c 和 timestamp，这也是 Qt 上位机主要解析的数据来源。MQTT 连接参数在代码中默认设置为 mqtt://192.168.127.152:1884，发布主题为 sensor/adxl345，用户名为 fangkuai，密码为 2120033，在线/离线状态主题为 sensor/adxl345/status，这些默认值均可在 publisher.cpp 中找到。

程序启动时会从命令行接收一个 QoS 参数，因此运行时可指定 0、1 或 2，用于测试不同消息质量等级。连接成功后，程序会先向状态主题发布一个保留的 online 消息，随后每隔两秒读取一次传感器和系统状态，并将 JSON 数据作为保留消息发布到主主题。当程序正常退出时，又会显式发送一个 offline 状态消息。这种设计便于上位机在重连后立即获取最近一次状态，也便于演示 MQTT 的遗嘱与保留消息机制。

树莓派终端还实现了 LED 联动控制。subscriber_led_wiringpi.cpp 中初始化了三路 LED 引脚，并在主循环中反复读取 pitch。程序根据 pitch 的大小将状态分为三个等级：当 pitch 小于 30 时认为是低等级，当 pitch 在 30 到 60 之间时认为是中等级，当 pitch 大于等于 60 时认为是高等级，并点亮不同 LED。这个逻辑虽然简单，但能够直观体现传感器数据驱动本地执行器的效果。

6. Qt 上位机设计

Qt 上位机是本系统面向用户的核心交互界面。程序入口位于 src/main.cpp，负责初始化 QApplication、加载图标、创建主窗口并启动事件循环。主界面类 MainWindow 定义于 include/mainwindow.h，实现于 src/mainwindow.cpp。

从功能上看，上位机主要包括五个部分。第一是连接参数区域，用于输入 Broker 地址、客户端 ID、用户名和密码。第二是订阅设置区域，用于配置数据主题、遗嘱主题和接收 QoS。第三是发布区域，允许用户直接输入 JSON 并发布到指定主题。第四是日志区域，用于显示连接状态、订阅情况、消息收发以及异常提示。第五是实时图形区域，用于显示当前选定指标的时间序列曲线，并给出最近 60 秒的平均值、

最小值和最大值。

上位机支持的主要指标包括 pitch、roll、x、y、z、cpu_temp 和 cpu_load，这一点可由 supportedMetricKeys() 看出。为提高兼容性，程序在解析 JSON 时并不只识别单一字段名，而是会按照多种可能路径查找对应数据。例如对于 X 轴加速度，既可识别 accel_x，也可识别 x 或带有层级路径的 sensor.x、adxl345.x 等；对于 CPU 温度，既可识别 cpu_temp_c，也可识别 cpu_temp 或 system.cpu_temp_c。这种设计使上位机对不同格式的数据源具有更好的适应性。

在可视化逻辑上，Qt 程序使用 QCustomPlot 绘制实时曲线，并为每个指标维护一段历史队列。代码中定义了 60 秒统计窗口和 600 秒历史缓存窗口，因此系统既能显示短时间内的统计值，也能保留较长时间的趋势数据。用户在界面中切换指标时，程序会从缓存中恢复对应曲线，而不是只显示切换之后新收到的数据，这提升了观察体验。

在 MQTT 功能上，上位机不仅可以订阅，也可以发布。publishJsonMessage() 函数会校验输入内容是否为合法 JSON，构造成紧凑格式后发送到指定主题。这样一来，Qt 上位机不仅是数据接收终端，也可以充当调试客户端，向树莓派终端发送测试消息。这对于演示双向通信和系统联调非常有帮助。

7. MQTT 通信机制说明

MQTT 是本项目的消息传输核心。系统中使用的 Broker 为 Mosquitto，客户端则包括树莓派终端发布者、树莓派终端 LED 订阅者、Qt 上位机以及命令行测试工具。

在主题设计上，主数据主题为 sensor/adxl345，状态主题为 sensor/adxl345/status。Qt 上位机默认订阅数据主题和遗嘱主题两个主题，从而同时获得实时数据和设备在线状态。树莓派终端发布 JSON 数据到主主题，发布 online/offline 状态到状态主题。根据代码，状态消息使用了保留标志，这意味着新加入的客户端在订阅后可以立即获得最近一次状态。

在 QoS 方面，树莓派发布程序通过命令行参数选择 0、1 或 2，而 Qt 上位机界面也支持订阅 QoS 和发布 QoS 的独立设置。这使系统可以直观演示不同 QoS 等级下的消息行为。

在遗嘱机制方面，树莓派终端和 Qt 上位机都配置了遗嘱消息。树莓派端在连接时将遗嘱主题设为 sensor/adxl345/status，遗嘱内容为 offline，QoS 为 1，且设置为保留消息。正常连接后，程序还会主动再发送一个 online 保留消息。这种方式能够区分“设备当前在线”和“设备意外断线”两类状态，对于上位机识别设备在线性非常实用。

8. JSON 数据格式说明

树莓派终端发布的 JSON 采用单条消息封装多项传感器与系统信息的方式，示意格式如下：

```
{
  "pitch": -31.87,
  "roll": -52.19,
  "accel_x": -129,
  "accel_y": -193,
  "accel_z": -76,
  "cpu_load": 0.12,
  "cpu_temp_c": 51.8,
  "timestamp": "2026-04-10T07:17:46Z"
}
```

其中，pitch 与 roll 表示姿态角，accel_x、accel_y、accel_z 为三轴原始加速度寄存器组合后的数值，cpu_load 为系统负载，cpu_temp_c 为摄氏温度，timestamp 为 UTC 时间戳。

Qt 上位机支持对这些字段进行解析和显示，尤其适合用来展示实时监测结果。由于程序本身还兼容更简化的字段名，如 x、y、z、cpu_temp 等，因此后续若替换为其他数据源，也只需在字段名上保持兼容即可。

9. 项目运行流程

本系统的运行流程可以概括为以下几个阶段。首先，在树莓派上完成 ADXL345 与 LED 的硬件连接，并确保 I^2C 和 GPIO 正常工作。其次，启动 Mosquitto Broker，并确认树莓派终端与 PC 上位机均能够访问该 Broker。然后，在树莓派终端启动 publisher.cpp 对应程序，指定 QoS 等级，使其开始周期性采集并发布 JSON 数据。接着，在 PC 上启动 Qt 上位机，输入 Broker 地址、用户名、密码、主题等参数并连接订阅。连接成功后，界面会实时显示当前指标的波形和状态信息。若需要测试本地执行器，则可在树莓派终端再运行 LED 联动程序，观察随姿态变化而变化的三色或三档 LED 状态。

10. 部署与使用说明

对于初学者而言，本项目建议按“先跑通 Broker，再跑树莓派采集，再跑 Qt 上位机”的顺序部署。

在 Windows 侧，需要先安装 Qt 5.14.2 和对应 MinGW 工具链，同时安装 Mosquitto 和 Paho MQTT C 库。仓库根目录 README 中已经给出了 Windows 环境的典型路径和 PowerShell 自检命令，可作为参考。Qt 工程通过 qmake 构建，工程文件为 EEN1071Ass2.pro。

在树莓派侧，需要准备好 Paho MQTT C、JsonCpp、WiringPi 以及 I^2C 相关开发环境。由于仓库中尚未提供现成的树莓派编译脚本，因此实际编译时需要根据系统安装情况手动编译。以 publisher.cpp 为例，可参考如下思路：将 publisher.cpp、ADXL345.cpp 和 I2CDevice.cpp 一起编译，并链接 paho-mqtt3c、jsoncpp 等库。需要注意的是，这里给出的是编译思路，而不是仓库内现成脚本，因此实际命令可能需要根据开发板环境调整。

如果只是用于联调或演示，也可以先在 Qt 上位机中使用自带发布功能，或者借助 MQTTX 与 Mosquitto 命令行工具先模拟数据，而不必一开始就完成整套树莓派环境配置。这一点对初学者尤其重要，因为先打通消息链路，再接入真实硬件，调试效率会更高。

11. 功能验证与实验现象

根据当前代码实现，本项目至少可验证以下几类功能。第一类是基础通信验证，即树莓派终端是否能够成功连接到 Broker 并发布 JSON 数据，Qt 上位机是否能够成功订阅并解析消息。第二类是 MQTT 特性验证，包括 QoS 0、QoS 1、QoS 2 的差异，以及遗嘱消息与保留消息的效果。第三类是图形界面验证，包括连接、断开、订阅、日志显示、实时曲线和 60 秒统计值更新。第四类是执行器验证，即 LED 是否能够根据 ADXL345 的倾斜程度切换状态。第五类是双向通信验证，即 Qt 上位机是否能够主动向主题发布 JSON，并被树莓派终端接收。

从工程展示角度看，这样的验证链条已经较完整，因为它不仅说明系统“能跑”，还说明系统“可观测、可交互、可联动”。

12. 项目特点与创新点

本项目的一个突出特点，是它并非单独完成某个环节，而是构建了完整的数据管线。从传感器采集、数据打包、消息传输、桌面显示到本地执行器联动，形成了一个闭环系统。对于课程项目而言，这比单纯读取一个传感器或单纯写一个上位机更具系统性。

第二个特点是工程实现较为贴近真实应用。树莓派终端不仅上传传感器数据，还上传自身 CPU 状态；Qt 上位机不仅支持接收，还支持主动发布；MQTT 不仅做基础收发，还引入了遗嘱消息、保留消息和 QoS 控制。这些都使项目从“演示型代码”更接近“可用型原型”。

第三个特点是可扩展性较好。当前系统以 ADXL345 为数据源，但 Qt 上位机的 JSON 解析逻辑具有兼容性，后续可以接入其他传感器，甚至接入多个主题的数据流，只需在字段映射与界面展示上做少量修改即可。

13. 结论

本项目实现了一套基于 Raspberry Pi 5、ADXL345、Mosquitto 和 Qt 的实时传感器数据采集与可视化系统。系统能够完成传感器姿态读取、加速度采集、系统状态读取、MQTT 消息发布、Qt 实时显示、统计分析以及 LED 联动控制，验证了从嵌入式采集端到桌面可视化端的完整技术链路。该项目具有较好的教学价值、展示价值和扩展潜力，适合作为物联网、嵌入式系统、MQTT 通信和上位机设计相关课程中的综合实验或原型系统。

使用导引

本项目分为两部分：Windows 上的 Qt 上位机和 Raspberry Pi 上的采集与控制程序。Qt 上位机负责连接 MQTT Broker、订阅主题、显示实时曲线和发送 JSON 测试消息；树莓派终端负责通过 I^2C 读取 ADXL345 数据，并以 JSON 形式发布到 MQTT 主题，同时可根据倾斜角控制 LED 状态。

首次使用时，建议先在 Windows 上安装 Mosquitto，启动本地或局域网 Broker，再运行 Qt 上位机测试连接。连接正常后，可以使用 MQTTX 或命令行工具向指定主题发送 JSON，确认界面能够正确接收和显示。完成这一阶段后，再部署树莓派终端程序，接入真实 ADXL345 和 LED，完成整套系统联调。

默认主题与默认参数可直接参考源码，其中 Qt 上位机默认 Broker、主题和鉴权参数定义在 mainwindow.h，树莓派终端默认参数定义在 publisher.cpp。如需修改网络环境、端口号、用户名密码或主题名称，可直接在上述文件中调整后重新编译。